RAG 모델 버전과 구조 설계

2026-10-14 | 팀 공유용 코드와 실행 기준

목적: Baseline은 기준 보존, Main은 팀 공동 개선, Version 1~4는 실제 코드가 포함된 참고 후보입니다. 기존 modular 저장소와 병합하지 않았습니다.

배포 기본: Claude Haiku 4.5 API - claude-haiku-4-5-20251001
실제 정리 수: 기준 1개 + 참고 4개 = 5개 구성; Main은 기준의 독립 작업 사본

GitHub 공개 공유 저장소 열기

버전별 차이

폴더	파서 / 청크	임베딩 / 검색	상태
Baseline	custom olefile / PyMuPDF fixed 1000 / 200	KURE-v1 / Chroma dense Top5 / 재순위 off	기준 코드 실행 가능
Main	Baseline과 초기 동일	초기 파일 35개 일치 독립 복사본 / 링크 없음	팀 공동 개선
Version 1	HwpHwpxLoader / Markdown PDF 700 / 100	BGE-m3-ko / FAISS business 식별 / BM25 RRF / Alibaba	실제 단계 코드 포함
Version 2	같은 파서 700 / 100	BGE-m3-ko / FAISS Kiwi BM25 RRF / Alibaba	실제 단계 코드 포함
Version 3	HWP 로더 보정 / Markdown PDF child 800 / 150 parent 8000	OpenAI 임베딩 / FAISS RRF / BGE 재순위 / parent Top4	실제 단계 코드 포함
Version 4	HWP 로더 보정 / PyPDFLoader 500 / 100	KoE5 / FAISS dense Top4 또는 RRF Top5	실제 단계 코드 포함

사용 범위: 사용자 추가 승인 후 네 원본 단계 코드 31개와 공유 부품 3개를 포함했습니다. 별도 원본 복사 없이 설치·선택·실행할 수 있습니다. 원본 SHA·AST·출처를 보존하며 번호는 성능 순위가 아닙니다. Main의 파일 수정은 Baseline 파일을 바꾸지 않습니다.

발표 자료는 기존 평가 시점 구성입니다. 배포 실행 기본은 Haiku 4.5이며 동일조건 품질 재평가 전입니다.

코드와 아키텍처

문서 준비 - Baseline



입력 문서를 파싱하고 출처 ID를 유지합니다. 청크·벡터·문서 요약은 로컬 데이터 폴더에 준비합니다. 원문·벡터·요약은 저장소에 올리지 않습니다.

질문 처리 - Baseline



질문 재작성, 분류, 요약, 비교 병합과 생성은 같은 Anthropic 공급자를 사용합니다. 약한 근거는 거절하고 근거 ID·숫자·날짜를 검사합니다. 이 규칙은 독립적인 의미 정답 판정을 보장하지 않습니다.

실제 모듈과 입출력

위치 / 역할	입력 → 출력
Baseline/rfp_rag/ingest/load.py parser	data_dir → 문서 dict 목록; ID·본문·파서 상태
ingest/chunk.py chunker	문서 dict + 크기/겹침 → 청크 dict 목록
rfp_rag/index.py embedder	cfg → 임베딩 공급자; 색인은 CLI로 별도 구축
rfp_rag/retrieve.py retriever	cfg + question → 근거 Document 목록
rfp_rag/pipeline.py / answer.py generator + verification	question + state + cfg → 답변·근거·found; 대화 상태 갱신
rfp_rag/providers.py 공급자 연결	cfg + 개인 환경 키 → ChatAnthropic; 구조화 결과를 기존 JSON 문자열 규약으로 변환
version_runtime.py / version_inputs.py 참고 버전 공통 입구	Version 폴더의 실제 단계 파일 + CLI 입력 → run/call 세션; Version_3/components 공유

팀원이 시작하는 방법

1. 버전과 키 설정

루트에서 python run.py --list로 Baseline/Main과 v1~v4를 확인합니다. 공동 개선은 Main 폴더에서 시작합니다. Main과 Baseline은 별도 프로세스로 실행하고 기본 data 경로도 각각 아래에 둡니다. .env.example 키는 비어 있으며 .env를 자동으로 읽지 않습니다.

```
cd Main
python -m venv .venv
.venv₩Scripts₩activate
python -m pip install -r requirements.txt

$secureKey = Read-Host "Anthropic API key" -AsSecureString
$env:ANTHROPIC_API_KEY = ([System.Net.NetworkCredential]::new("", $secureKey)).Password
```

2. 데이터 준비와 실행

data/raw에 본인 문서와 data_list.csv를 둡니다. 준비한 KURE 모델·색인·요약이 필요합니다. 색인 구축은 모델이 없으면 다운로드할 수 있고 요약·질문 호출은 API 비용이 발생합니다. 이 작업에서 다운로드와 API 호출은 0회입니다.

```
python -m rfp_rag.ingest.chunk --exp 07_verify
python -m rfp_rag.index --exp 07_verify --embedder nlpai-lab/KURE-v1 --method fixed
python -m rfp_rag.ingest.enrich --exp 07_verify
python -m rfp_rag.pipeline --exp 07_verify --ask "사업 요구사항을 설명해 줘."
```

3. 후보의 실행 연결과 평가 한계

참고 후보는 루트에서 pip install -r Version_1/requirements.txt 후 python Version_1/run.py --smoke 또는 --input-dir data/raw --metadata data/data_list.csv --ask 질문으로 실행합니다. Version 2도 같고 Version 3~4는 metadata 없이 실행합니다. 각 폴더에 실제 parser/chunking/embedding/retrieval/generation과 원본 검수 코드가 있습니다.

Version 1~2: parser → cleaning → chunker → embedder → Haiku → retriever → generator. Version 3~4: HWP 보정 → parser → chunker → embedder → Haiku → generator → retriever. CLI가 실제 라이브러리·입력을 준비하며 Version 1~2의 CSV에는 공고 차수도 필요합니다. Version 3의 OpenAI 임베딩용 키는 별도 준비합니다. 생성 공급자만 Haiku로 변경했고 기본 출력 상한은 1500토큰입니다.

검증: 네 Version 엔트리포인트에서 원본 청킹·BM25·RRF·부모 확장·재순위·문맥 구성 코드를 실행했습니다. 로더·모델 공급자만 합성하며 API 호출은 0회입니다. 고정 소스·AST와 Main 35파일 동일성을 확인했습니다. 실제 API 모델 품질·GPU 전체 실행·새 설치 환경은 미검증입니다.

근거: 기존 07_verify는 대체 후보 전체의 개선이 입증되지 않아 유지한 기준입니다. 과거 EXAONE·Luna 관측과 Haiku 실행 설정을 구분하며 같은 조건의 전체 순위는 보류합니다. 최신 포함 승인·출처는 SOURCE-NOTES.md, 기존 평가 근거는 Baseline/docs, 발표 원본은 presentations에 보존합니다.